Labo 5 Geavanceerde Computerarchitectuur

Arne Van der Stede en Syme Vandenbosch

Academiejaar 2025-2026
March 26, 2026

1 Introductie

In dit labo wordt er gekeken naar het prestatieverschil van asynchrone GPU-uitvoering ten opzichte van synchrone uitvoering. Bij synchrone uitvoering blokkeert de CPU telkens totdat de GPU klaar is, waardoor kostbare tijd verloren gaat. Door gebruik te maken van asynchrone operaties kunnen de CPU en GPU parallel werken waardoor de totale uitvoeringstijd kleiner wordt. In dit labo wordt er gekeken naar het verschil in uitvoeringstijd tussen een synchrone en een asynchrone aanpak bij het uitvoeren van reductiebewerkingen (som, product, minimum en maximum) op vier onafhankelijke arrays.
Door de totale uitvoeringstijden van beide benaderingen te vergelijken kunnen we inzicht krijgen in de tijdswinst die asynchrone uitvoering kan opleveren. De tijden worden gemeten met de Chrono-bibliotheek op de CPU, zodat ook de overhead van de CPU-code meegeteld wordt in de vergelijking.

2 Code

```
#include <iostream>
#include <cuda_runtime.h>
#include <cstdlib>
#include <cfloat>
#include <iomanip>
#include <vector>
#include <chrono>
#include <cmath>

enum Op { SUM, PROD, MIN, MAX };
const char* op_names[] = {"SUM", "PROD", "MIN", "MAX"};

void generate_data(float* data, int n, int op) {
    for (int i = 0; i < n; ++i) {
        if (op == PROD) {
            data[i] = 1.0f - (i % 1000) / 50000.0f; // values in (0, 1]
        } else {
            data[i] = (i % 100) / 10.0f;
        }
    }
}

float finalize_reduction_cpu(float* results, int n, int op) {
    float res;
    if (op == SUM) {
        res = 0.0f;
        for (int i = 0; i < n; i++) res += results[i];
    } else if (op == PROD) {
        res = 1.0f;
        for (int i = 0; i < n; i++) res *= results[i];
    } else if (op == MIN) {
        res = FLT_MAX;
        for (int i = 0; i < n; i++) if (results[i] < res) res = results[i];
    } else {
        res = -FLT_MAX;
        for (int i = 0; i < n; i++) if (results[i] > res) res = results[i];
    }
    return res;
}

template <Op op>
__global__ void reduce_kernel(const float *in, float *out, int N) {
    extern __shared__ float sdata[];
    int tid = threadIdx.x;
```

```
    int idx = blockIdx.x * blockDim.x + tid;

    float identity;
    if      (op == SUM)  identity = 0.0f;
    else if (op == PROD) identity = 1.0f;
    else if (op == MIN)  identity = FLT_MAX;
    else                 identity = -FLT_MAX;

    sdata[tid] = (idx < N) ? in[idx] : identity;
    __syncthreads();

    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            if      (op == SUM)  sdata[tid] += sdata[tid + stride];
            else if (op == PROD) sdata[tid] *= sdata[tid + stride];
            else if (op == MIN)  sdata[tid] = fminf(sdata[tid], sdata[tid + stride]);
            else                 sdata[tid] = fmaxf(sdata[tid], sdata[tid + stride]);
        }
        __syncthreads();
    }

    if (tid == 0) out[blockIdx.x] = sdata[0];
}

void launch_kernel(Op op, int gridSize, int BLOCK_SIZE, size_t sharedMemSize,
                   cudaStream_t stream, float* d_in, float* d_out, int N) {
    switch (op) {
        case SUM:  reduce_kernel<SUM> <<<gridSize, BLOCK_SIZE, sharedMemSize,
            stream>>>(d_in, d_out, N); break;
        case PROD: reduce_kernel<PROD><<<gridSize, BLOCK_SIZE, sharedMemSize,
            stream>>>(d_in, d_out, N); break;
        case MIN:  reduce_kernel<MIN> <<<gridSize, BLOCK_SIZE, sharedMemSize,
            stream>>>(d_in, d_out, N); break;
        case MAX:  reduce_kernel<MAX> <<<gridSize, BLOCK_SIZE, sharedMemSize,
            stream>>>(d_in, d_out, N); break;
    }
}

// Synchronous
double run_sequentially(long long N, int BLOCK_SIZE, int retries) {
    const Op ops[] = {SUM, PROD, MIN, MAX};
    int gridSize   = (N + BLOCK_SIZE - 1) / BLOCK_SIZE;
    size_t shMemSz = BLOCK_SIZE * sizeof(float);

    float *h_in, *h_out;
    cudaMallocHost(&h_in,  N        * sizeof(float));
    cudaMallocHost(&h_out, gridSize * sizeof(float));

    float *d_in, *d_out;
    cudaMalloc(&d_in,  N        * sizeof(float));
    cudaMalloc(&d_out, gridSize * sizeof(float));

    auto start = std::chrono::high_resolution_clock::now();

    for (int it = 0; it < retries; it++) {
        for (int i = 0; i < 4; i++) {
            generate_data(h_in, (int)N, ops[i]);
            cudaMemcpy(d_in, h_in, N * sizeof(float), cudaMemcpyHostToDevice);
            launch_kernel(ops[i], gridSize, BLOCK_SIZE, shMemSz, 0, d_in, d_out, (int)N);
            cudaMemcpy(h_out, d_out, gridSize * sizeof(float), cudaMemcpyDeviceToHost);
            finalize_reduction_cpu(h_out, gridSize, ops[i]);
        }
    }

    cudaDeviceSynchronize();
    auto end = std::chrono::high_resolution_clock::now();

    cudaFree(d_in);  cudaFree(d_out);
    cudaFreeHost(h_in); cudaFreeHost(h_out);

    return std::chrono::duration<double, std::milli>(end - start).count() / retries;
}

// Asynchronous
double run_overlapped(long long N, int BLOCK_SIZE, int retries) {
    const Op ops[] = {SUM, PROD, MIN, MAX};
    int gridSize   = (N + BLOCK_SIZE - 1) / BLOCK_SIZE;
    size_t shMemSz = BLOCK_SIZE * sizeof(float);
```

```
119
120     cudaStream_t streams[4];
121     cudaEvent_t  kernel_started[4];   // fired just before each kernel launch
122     float *h_data[4], *d_data[4];
123     float *h_res[4],  *d_res[4];
124
125     for (int i = 0; i < 4; i++) {
126         cudaStreamCreate(&streams[i]);
127         cudaEventCreate(&kernel_started[i]);
128         cudaMallocHost(&h_data[i], N        * sizeof(float));
129         cudaMallocHost(&h_res[i],  gridSize * sizeof(float));
130         cudaMalloc    (&d_data[i], N        * sizeof(float));
131         cudaMalloc    (&d_res[i],  gridSize * sizeof(float));
132     }
133
134     auto start = std::chrono::high_resolution_clock::now();
135
136     for (int it = 0; it < retries; it++) {
137         for (int i = 0; i < 4; i++) {
138             generate_data(h_data[i], (int)N, ops[i]);
139
140             if (i > 0)
141                 cudaStreamWaitEvent(streams[i], kernel_started[i - 1], 0);
142
143             cudaMemcpyAsync(d_data[i], h_data[i],
144                             N * sizeof(float), cudaMemcpyHostToDevice, streams[i]);
145
146             cudaEventRecord(kernel_started[i], streams[i]);
147             launch_kernel(ops[i], gridSize, BLOCK_SIZE, shMemSz,
148                           streams[i], d_data[i], d_res[i], (int)N);
149
150             cudaMemcpyAsync(h_res[i], d_res[i],
151                             gridSize * sizeof(float), cudaMemcpyDeviceToHost,
                                    streams[i]);
152         }
153
154         cudaDeviceSynchronize();
155
156         for (int i = 0; i < 4; i++)
157             finalize_reduction_cpu(h_res[i], gridSize, ops[i]);
158     }
159
160     auto end = std::chrono::high_resolution_clock::now();
161
162     for (int i = 0; i < 4; i++) {
163         cudaStreamDestroy(streams[i]);
164         cudaEventDestroy(kernel_started[i]);
165         cudaFree(d_data[i]); cudaFree(d_res[i]);
166         cudaFreeHost(h_data[i]); cudaFreeHost(h_res[i]);
167     }
168
169     return std::chrono::duration<double, std::milli>(end - start).count() / retries;
170 }
171
172 int main() {
173     const int      BLOCK_SIZE = 512;
174     const long long startN    = 10;
175     const long long endN      = 100000000;
176     const int      steps      = 25;
177     const int      retries      = 20;
178
179     double factor = std::pow((double)endN / startN, 1.0 / (steps - 1));
180
181     // Warmup
182     run_sequentially(100, BLOCK_SIZE, 1);
183     run_overlapped  (100, BLOCK_SIZE, 1);
184
185     std::cout << std::setw(12) << "N Elements" << " | "
186               << std::setw(15) << "Sequential (ms)" << " | "
187               << std::setw(15) << "Overlapped (ms)" << " | "
188               << std::setw(10) << "Speedup"  << "\n";
189     std::cout << std::string(60, '-') << "\n";
190
191     for (int i = 0; i < steps; i++) {
192         long long N = static_cast<long long>(startN * std::pow(factor, i));
193
194         double seq_time = run_sequentially(N, BLOCK_SIZE, retries);
195         double ovl_time = run_overlapped  (N, BLOCK_SIZE, retries);
```

```
196
197         std::cout << std::setw(12) << N << " | "
198                   << std::setw(15) << std::fixed << std::setprecision(3) << seq_time <<
                        " | "
199                   << std::setw(15) << std::fixed << std::setprecision(3) << ovl_time <<
                        " | "
200                   << std::setw(10) << std::fixed << std::setprecision(2)
201                   << (ovl_time > 0 ? seq_time / ovl_time : 1.0) << "x\n";
202     }
203
204     return 0;
205 }
```

Listing 1: reduction

3 Resultaten

3.1 Terminal uitvoer

N Elements	Sequential (ms)	Overlapped (ms)	Speedup
10	1.099	0.846	1.30x
19	1.273	0.441	2.89x
38	0.686	0.694	0.99x
74	0.673	0.602	1.12x
146	1.250	0.420	2.98x
287	0.722	0.461	1.57x
562	1.006	0.435	2.31x
1100	0.672	0.504	1.33x
2154	0.910	0.433	2.10x
4216	1.162	0.951	1.22x
8254	1.011	0.599	1.69x
16155	1.668	0.974	1.71x
31622	1.449	0.843	1.72x
61896	1.851	1.677	1.10x
121152	2.833	1.978	1.43x
237137	4.796	3.202	1.50x
464158	8.037	6.003	1.34x
908517	14.173	12.196	1.16x
1778279	25.226	20.397	1.24x
3480700	48.494	48.292	1.00x
6812920	90.780	77.242	1.18x
13335214	198.127	148.323	1.34x
26101572	371.991	339.136	1.10x
51089697	873.113	628.444	1.39x
99999999	1483.420	1399.259	1.06x

Figure 1: Resultaat van de code

3.2 Grafiek

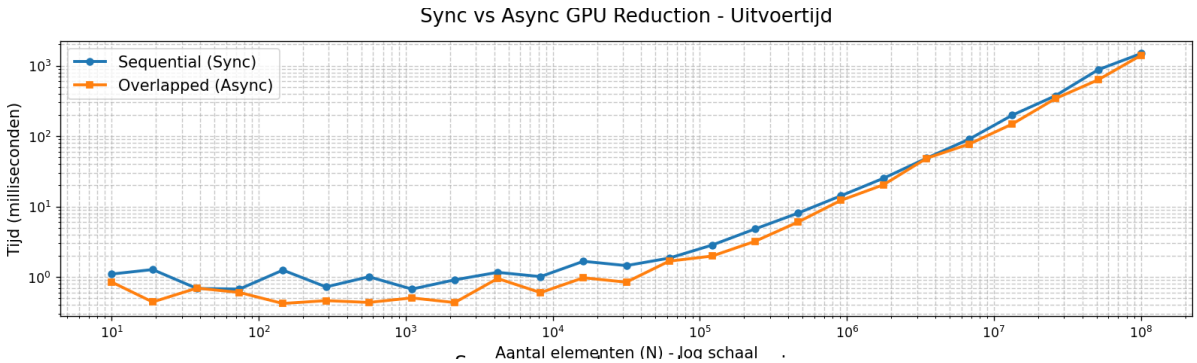


Figure 2: Grafiek van de uitvoeringstijd synchroon vergeleken met asynchroon

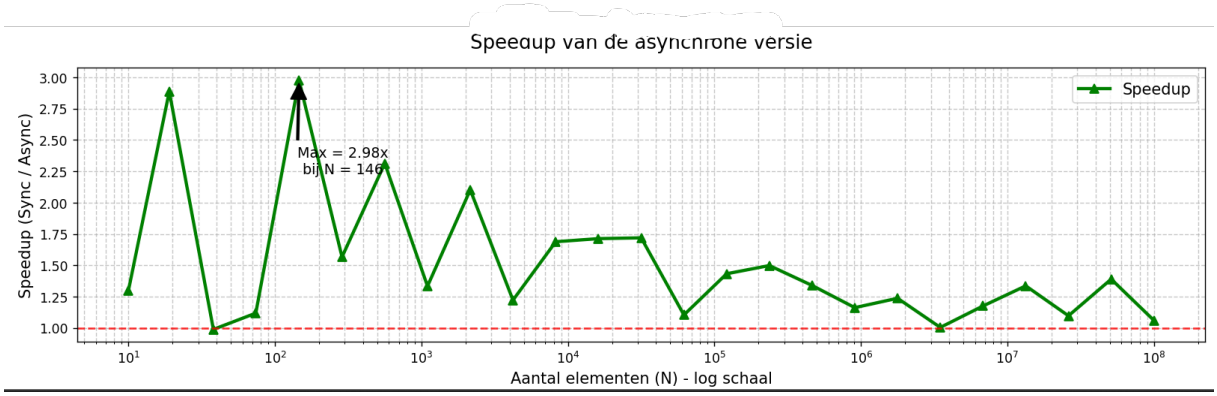


Figure 3: Grafiek van de speedup bij asynchrone versie

3.3 Besluiten

De metingen tonen aan dat asynchrone uitvoering een duidelijke snelheidswinst oplevert ten opzichte van de sequentiële aanpak. Bij kleine en middelmatige arrays (N tussen de 1 000 0000) is de speedup het grootst. binnen deze range van arrays wordt een maximale winst geboekt van bijna 3x speedup. Dit bewijst dat overlap tussen CPU-werk en GPU-werk mogelijk is.

Naarmate het aantal elementen toeneemt, daalt de speedup. Bij arrays groter dan 10 000 000 elementen variërt de snelheidswinst tussen de 1,1x en 1,5x. Bij de zeer grote arrays is de tijdswinst maar miniem. Deze afname in speedup heeft twee oorzaken:

- De reductiekernel is efficiënt, hierdoor is de rekentijd op de GPU heel kort. Hierdoor worden de PCIe-datatransfers en de CPU-generatietijd de bottleneck. De kernel is zo snel, waardoor de GPU zeer snel klaar is met één operatie. Op het moment dat de GPU kaar is zit hij gewoon te wachten tot de CPU de volgende array heeft gegenereerd.

  Wat als gevolg heeft dat er te weinig "werk" op de GPU is om te overlappen met het werk dat de CPU moet doen. Waardoor de asynchrone versie bijna geen winst meer oplevert.

- Bij grote arrays ontstaat concurrentie op de PCIe-bus en de copy engines van de GPU. Wanneer vier streams tegelijk data overbrengen beperkt dit de effectieve overlap. De transfers starten ongeveer tegelijk, waardoor de PCIe-bus overbelast raakt. De copy engines moeten hierdoor de transfers serialiseren of vertragen. Het totale transfer-volume per iteratie is 4x wat leidt tot zeer grote hoeveelheid data die over de bus moet.

  Wat als gevolg heeft dat hoewel er theoretisch naar overlap gestreefd wordt. Wordt de effectieve snelheid van de transfers langer dan wanneer ze één voor één uitgevoerd zouden worden (sequentiële werking). Hierdoor verdwijnt er een groot deel van de verwachte tijdswinst van de asynchrone uitvoering.

We kunnen besluiten dat asynchrone uitvoering met streams voordelen biedt. Maar de winst blijft wel beperkt, zeker bij zeer grote arrays.

4 Conclusie

Deze labo-opdracht had als doel de verschillen tussen synchrone en asynchrone uitvoering op de GPU te onderzoeken aan de hand van reductie-operaties (sum, product, min en max) op vier onafhankelijke arrays.

Uit de resultaten blijkt dat de gebruikte aanpak hiervoor werkt. Vooral voor kleine tot middelgrote datasets. Echter wordt het ook duidelijk dat asynchrone uitvoering niet altijd een lineaire speedup oplevert. Vooral niet wanneer datatransfers of de data-generatie van de CPU doorslaggevend worden. Dit maakt duidelijk dat de PCIe-bandbreedte en CPU-GPU overlap vaak de werkelijke limieten bepalen.

Er kan gezegd worden dat asynchrone GPU-computing een krachtig instrument is, maar het wel een goed begrip van de hardware-beperkingen vraagt en een zorgvuldige afstemming van CPU- en GPU-werk om het maximale rendement te kunnen halen.